

Linux Class 7 - Task

Reading Files, Checking Groups, Memory and Links

Submitted by: Shivakumar Muradi Date: 29 September 2026

For this task I made a folder called class7 and inside it a small file called notes.txt with five lines, which I typed in vi. For the longer file I did not make one myself, I used /etc/services, which is a file that is already there in Ubuntu and has thousands of lines in it. That was useful because more and less only show their difference when the file is too long for one screen.

1. What is the difference between more, less and cat?

All three of them show what is inside a file. The difference is how much they show at one time. This is the short way I remember it:

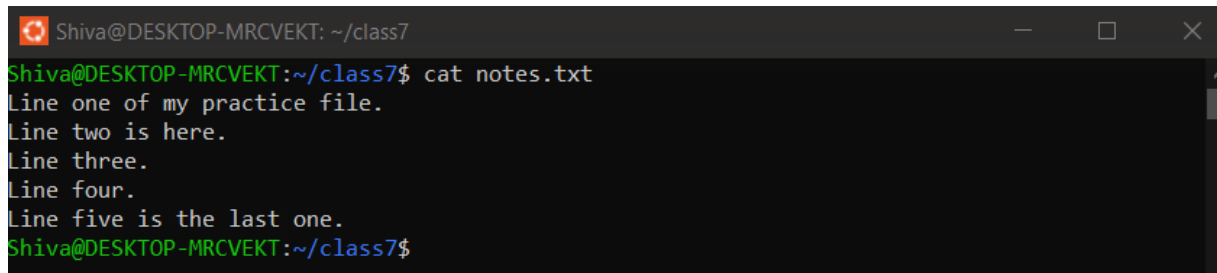
Command	How it shows the file	When I would use it
cat	Everything at once, no stopping.	Small files, two or three lines.
more	One screen, then waits. Only goes forward.	A long file I just want to read from the top.
less	One screen, then waits. Goes forward and backward.	A long file where I need to move around or search.

Now the same thing explained one by one, with what I saw on my screen.

cat

cat prints the whole file on the screen in one go and then gives the prompt back. It does not stop anywhere.

```
cat notes.txt
```



Screenshot 1: cat shows the whole of my small file at once.

This is fine for my notes.txt because it is only five lines. But if the file has a thousand lines, everything scrolls past and I can only see the end of it. That is the problem the other two commands solve.

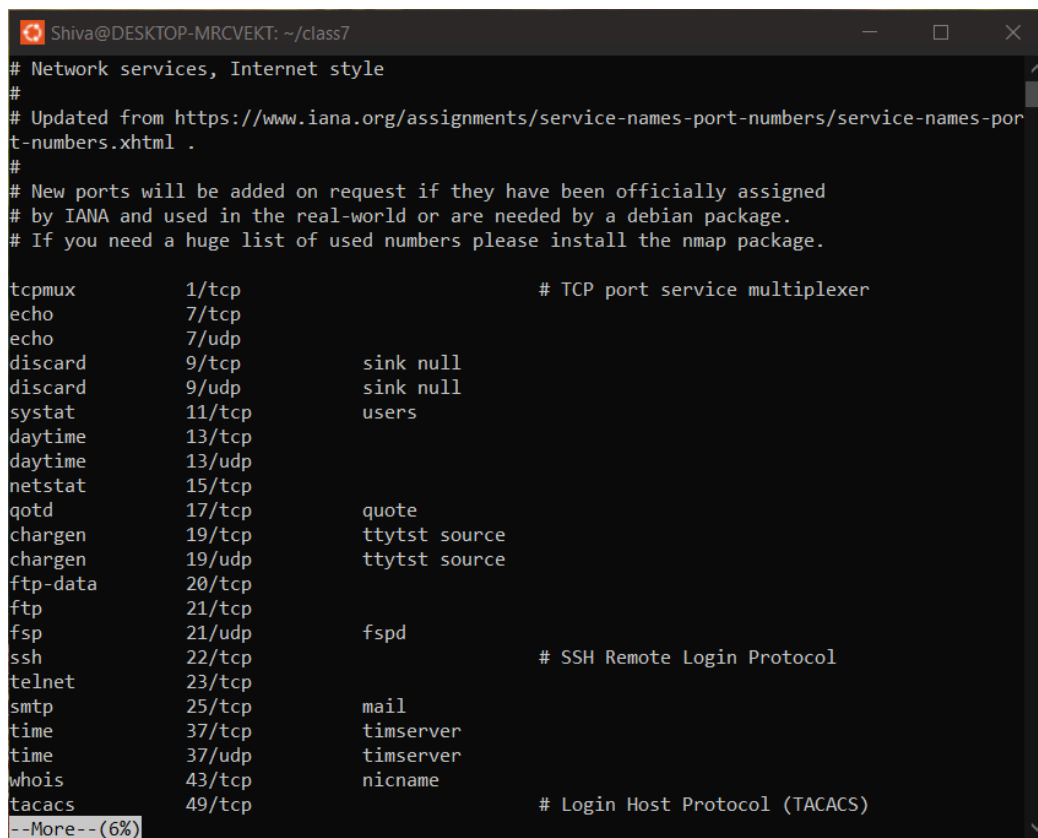
more

more shows one screenful and then stops and waits. At the bottom it writes --More-- with a percentage, so I know how much of the file I have already seen.

```
more /etc/services
```

- Space bar shows the next screen.
- Enter shows just one more line.
- q quits and comes back to the prompt.

The thing about more is that it only goes forward. Once I press space I cannot go back up to the line I just passed.



```
Shiva@DESKTOP-MRCVEKT: ~/class7
# Network services, Internet style
#
# Updated from https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml .
#
# New ports will be added on request if they have been officially assigned
# by IANA and used in the real-world or are needed by a debian package.
# If you need a huge list of used numbers please install the nmap package.

tcpmux      1/tcp                # TCP port service multiplexer
echo        7/tcp
echo        7/udp
discard     9/tcp              sink null
discard     9/udp              sink null
systat      11/tcp             users
daytime     13/tcp
daytime     13/udp
netstat     15/tcp
qotd        17/tcp              quote
chargen     19/tcp             ttytst source
chargen     19/udp             ttytst source
ftp-data    20/tcp
ftp         21/tcp
fsp         21/udp             fspd
ssh         22/tcp             # SSH Remote Login Protocol
telnet      23/tcp
smtp        25/tcp             mail
time        37/tcp             timserver
time        37/udp             timserver
whois       43/tcp             nicname
tacacs      49/tcp             # Login Host Protocol (TACACS)
--More--(6%)
```

Screenshot 2: more stops after one screen and shows --More--(6%) at the bottom.

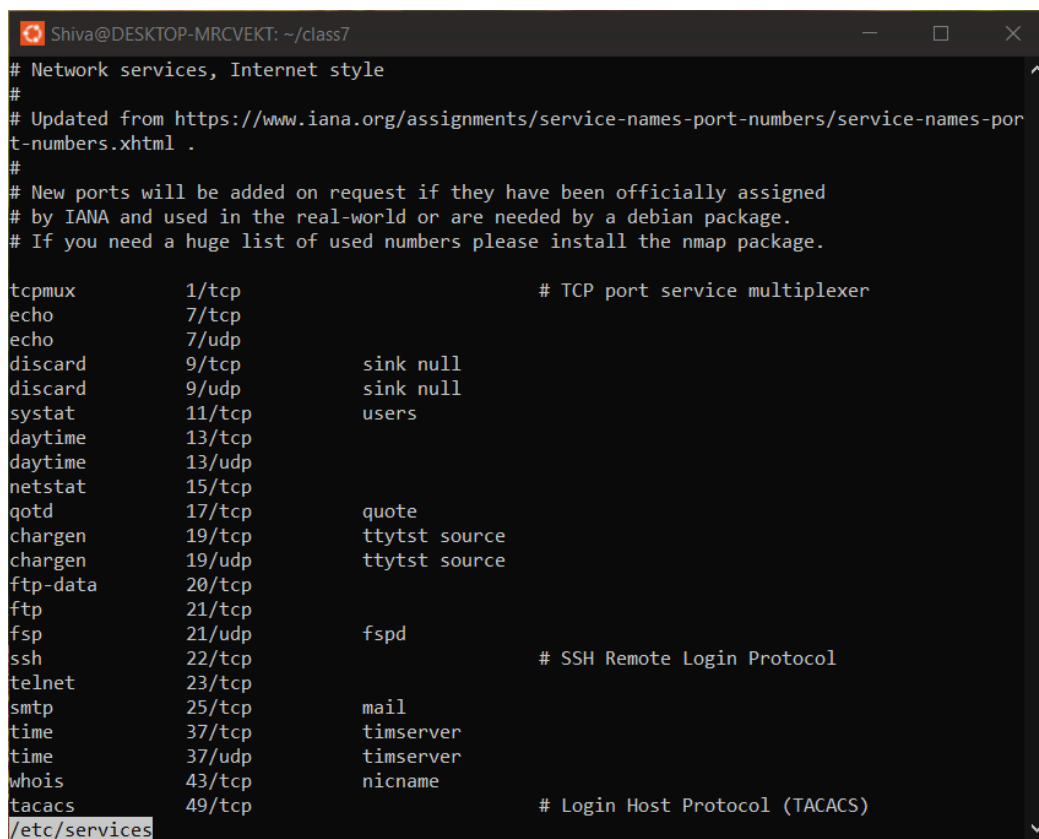
less

less also shows one screenful at a time, but it is better because it can move in both directions.

```
less /etc/services
```

- Arrow keys move up and down one line.
- Space bar goes a full page down, b goes a page back.
- /word searches for a word inside the file.
- q quits.

At the bottom less writes the name of the file instead of a percentage. Another thing RK sir mentioned is that less does not read the whole file before showing it, so a very big file opens straight away.



```
Shiva@DESKTOP-MRCVEKT: ~/class7
# Network services, Internet style
#
# Updated from https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml .
#
# New ports will be added on request if they have been officially assigned
# by IANA and used in the real-world or are needed by a debian package.
# If you need a huge list of used numbers please install the nmap package.

tcpmux      1/tcp                # TCP port service multiplexer
echo        7/tcp
echo        7/udp
discard     9/tcp              sink null
discard     9/udp              sink null
systat      11/tcp             users
daytime     13/tcp
daytime     13/udp
netstat     15/tcp
qotd        17/tcp              quote
chargen     19/tcp             ttytst source
chargen     19/udp             ttytst source
ftp-data    20/tcp
ftp         21/tcp
fsp         21/udp             fspd
ssh         22/tcp             # SSH Remote Login Protocol
telnet      23/tcp
smtp        25/tcp             mail
time        37/tcp             timserver
time        37/udp             timserver
whois       43/tcp             nicname
tacacs      49/tcp             # Login Host Protocol (TACACS)

/etc/services
```

Screenshot 3: less shows the same file and writes the file name at the bottom.

There is also a saying about it, that less is more than more, because less can do everything more can do and some extra things as well.

2. How to check which usergroup a user belongs to?

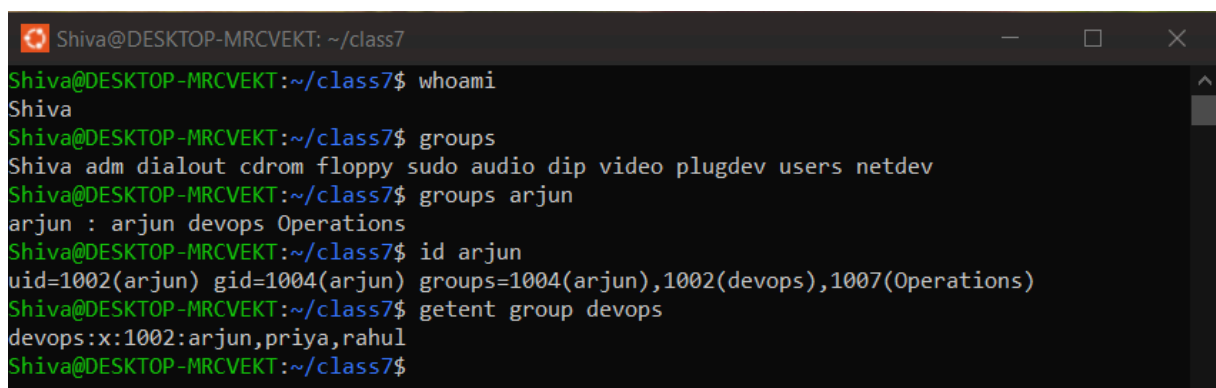
The command is groups and after it I write the user name.

```
groups arjun
```

The answer came as arjun : arjun devops Operations. The first name after the colon is his main group, the one he gets automatically, and the names after that are the extra groups he was added to.

If I write groups with no name at all, it shows my own groups.

```
groups
```



```
Shiva@DESKTOP-MRCVEKT: ~/class7
Shiva@DESKTOP-MRCVEKT:~/class7$ whoami
Shiva
Shiva@DESKTOP-MRCVEKT:~/class7$ groups
Shiva adm dialout cdrom floppy sudo audio dip video plugdev users netdev
Shiva@DESKTOP-MRCVEKT:~/class7$ groups arjun
arjun : arjun devops Operations
Shiva@DESKTOP-MRCVEKT:~/class7$ id arjun
uid=1002(arjun) gid=1004(arjun) groups=1004(arjun),1002(devops),1007(Operations)
Shiva@DESKTOP-MRCVEKT:~/class7$ getent group devops
devops:x:1002:arjun,priya,rahul
Shiva@DESKTOP-MRCVEKT:~/class7$
```

Screenshot 4: groups on its own, groups with a user name, and id for the same user.

id arjun gives the same information but with the numbers as well, like uid=1002(arjun) and gid=1004(arjun). The uid is the number of the user and the gid is the number of his main group.

getent group devops asks the opposite question. Instead of asking which groups one user is in, it shows who is inside one group. For devops it printed devops:x:1002:arjun,priya,rahul, so the names after the last colon are the members.

The same information is kept in a file called /etc/group, and I can read it with cat /etc/group if I want to see all the groups together.

3. Explore the free and free -h command

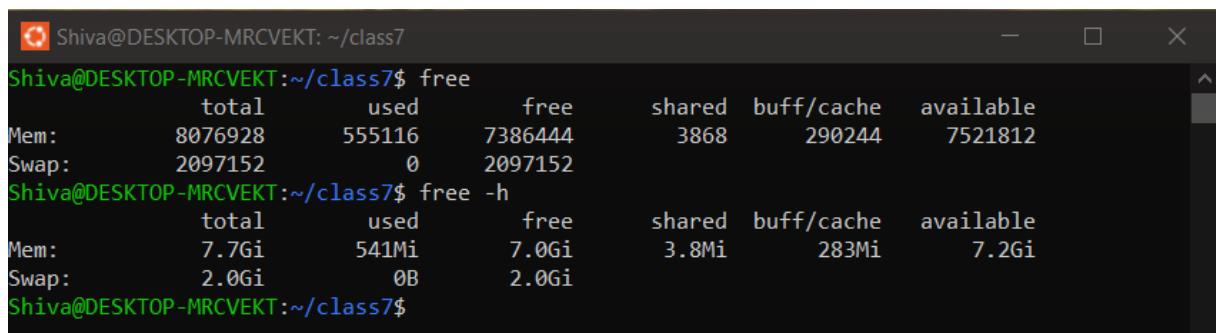
free shows how much memory, that is RAM, the system has and how much of it is being used right now.

```
free
```

On my machine it printed the total as 8076928. That number is in kilobytes, which is hard to read. So there is an option to make it easier:

```
free -h
```

Now the same line says 7.7Gi. The h stands for human readable, which means it changes the numbers into GB and MB so a person can understand them quickly.



```
Shiva@DESKTOP-MRCVEKT: ~/class7
Shiva@DESKTOP-MRCVEKT:~/class7$ free
              total        used        free      shared  buff/cache   available
Mem:      8076928       555116       7386444        3868       290244       7521812
Swap:      2097152           0         2097152
Shiva@DESKTOP-MRCVEKT:~/class7$ free -h
              total        used        free      shared  buff/cache   available
Mem:          7.7Gi         541Mi         7.0Gi         3.8Mi         283Mi         7.2Gi
Swap:          2.0Gi           0B          2.0Gi
Shiva@DESKTOP-MRCVEKT:~/class7$
```

Screenshot 5: the same memory shown in kilobytes and then in GB.

The columns mean:

- total is all the RAM in the machine.
- used is how much is being used by programs.
- free is how much is completely empty.
- shared is memory that more than one program is using together.
- buff/cache is memory Linux is borrowing to keep recently used files handy.
- available is how much a new program could actually get.

The part I did not understand at first is why free and available are different numbers. Linux keeps old files in the buff/cache memory so that it does not have to read them from the disk again. That memory looks busy, but if a new program needs it, Linux gives it away immediately. So available is the honest number to look at, not free.

The second line is Swap. Swap is space on the hard disk that is used like extra memory when the RAM gets full. On my machine it shows 2.0Gi total and 0B used, which means it has not been needed.

A few more options I tried:

- free -m shows the numbers in MB and free -g shows them in GB.
- free -t adds one more line at the bottom with the totals.
- free -s 2 keeps printing the memory again every 2 seconds, until I press Ctrl+C.

4. What does `sudo chgrp <groupname> <file-name>` do?

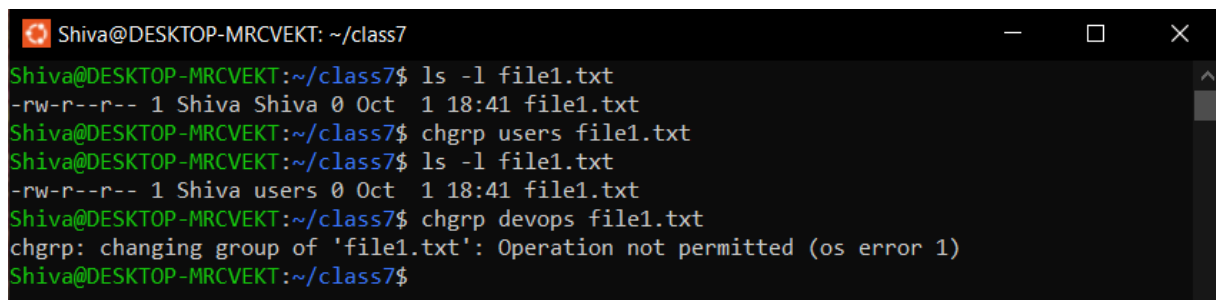
`chgrp` is short for change group. It changes which group a file belongs to. The owner of the file is not touched, only the group.

```
sudo chgrp devops file1.txt
```

To try it I made an empty file called `file1.txt`. Before changing anything, `ls -l` showed Shiva Shiva. The first name is the owner and the second name is the group, so the file belonged to me and to my own group. Then I changed the group:

```
chgrp users file1.txt
```

Now the same line says Shiva users. Only the second name changed, which proves that `chgrp` touches the group and leaves the owner alone.

A screenshot of a terminal window titled 'Shiva@DESKTOP-MRCVEKT: ~/class7'. The terminal shows the following commands and output:

```
Shiva@DESKTOP-MRCVEKT:~/class7$ ls -l file1.txt
-rw-r--r-- 1 Shiva Shiva 0 Oct 1 18:41 file1.txt
Shiva@DESKTOP-MRCVEKT:~/class7$ chgrp users file1.txt
Shiva@DESKTOP-MRCVEKT:~/class7$ ls -l file1.txt
-rw-r--r-- 1 Shiva users 0 Oct 1 18:41 file1.txt
Shiva@DESKTOP-MRCVEKT:~/class7$ chgrp devops file1.txt
chgrp: changing group of 'file1.txt': Operation not permitted (os error 1)
Shiva@DESKTOP-MRCVEKT:~/class7$
```

Screenshot 6: the group changes from Shiva to users, and the second attempt is refused.

I did not need `sudo` for that one, because I am already a member of the users group. To show why the question has `sudo` in it, I then tried a group I am not a member of:

```
chgrp devops file1.txt
```

It refused with `chgrp: changing group of 'file1.txt': Operation not permitted`. Linux only lets me hand a file to a group I belong to myself. For any other group it is an administrator job, so the command has to be written as `sudo chgrp devops file1.txt` and then it asks for my password.

This is useful when a few people are working on the same file. If the file belongs to the devops group and everybody in that team is in the devops group, then all of them can work on it, without me having to give permission to each person one by one.

Two more things about it:

- `sudo chown :devops file1.txt` does exactly the same thing. The empty space before the colon means do not change the owner.
- `sudo chgrp -R devops myfolder` changes the group of a folder and everything inside it. The `R` means recursive.

I can always check the result with `ls -l`. In that line the first name is the owner and the second name is the group.

5. Can we create a link for files and directories?

Yes, we can. The command is `ln`, which is short for link. A link is one more name for a file that is already there. There are two kinds of links and they behave differently.

Hard link

A hard link is made without any option:

```
ln data.txt hardlink.txt
```

Both names now point to exactly the same file on the disk. To see this I used ls with the i option, which shows the inode number in front of every file. The inode number is like the address of the file on the disk.

```
ls -li
```

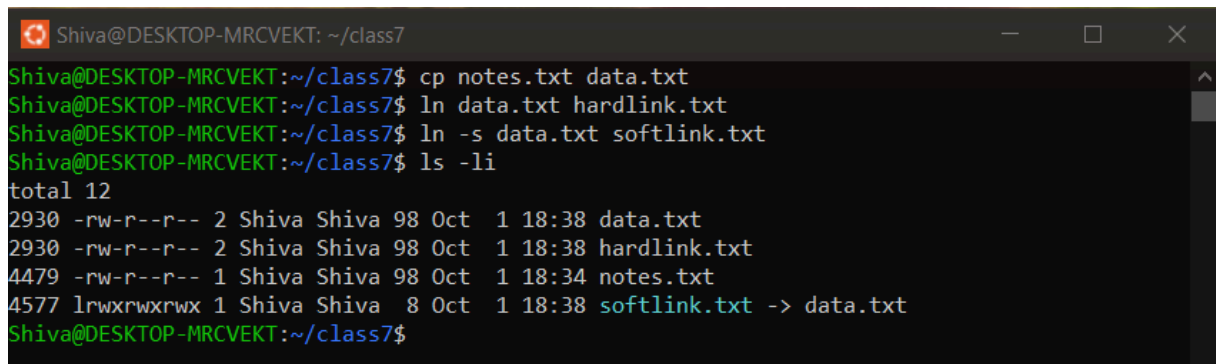
data.txt and hardlink.txt both showed the same inode number, 2930. The number just after the permissions also changed from 1 to 2. That number counts how many names this one file has.

Soft link

A soft link, also called a symbolic link, is made with the -s option:

```
ln -s data.txt softlink.txt
```

This one is not the same file. It is a small separate file that only holds the name of the original. In the listing it has a different inode number, its size is only 8 bytes, the line starts with the letter l instead of a dash, and it shows an arrow pointing to data.txt.



```
Shiva@DESKTOP-MRCVEKT: ~/class7
Shiva@DESKTOP-MRCVEKT:~/class7$ cp notes.txt data.txt
Shiva@DESKTOP-MRCVEKT:~/class7$ ln data.txt hardlink.txt
Shiva@DESKTOP-MRCVEKT:~/class7$ ln -s data.txt softlink.txt
Shiva@DESKTOP-MRCVEKT:~/class7$ ls -li
total 12
2930 -rw-r--r-- 2 Shiva Shiva 98 Oct  1 18:38 data.txt
2930 -rw-r--r-- 2 Shiva Shiva 98 Oct  1 18:38 hardlink.txt
4479 -rw-r--r-- 1 Shiva Shiva 98 Oct  1 18:34 notes.txt
4577 lrwxrwxrwx 1 Shiva Shiva  8 Oct  1 18:38 softlink.txt -> data.txt
Shiva@DESKTOP-MRCVEKT:~/class7$
```

Screenshot 7: the same inode number for the hard link, and a separate one with an arrow for the soft link.

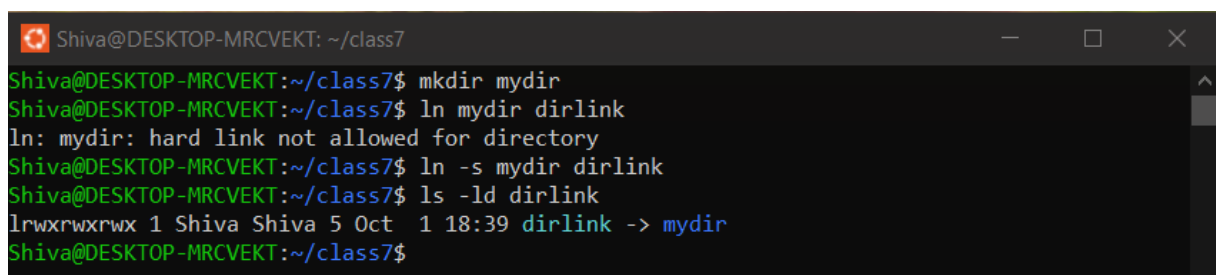
Links for directories

For a directory only the soft link works. When I tried to make a hard link of my folder, Linux refused:

```
ln mydir dirlink
```

It answered ln: mydir: hard link not allowed for directory. But with -s it worked without any complaint:

```
ln -s mydir dirlink
```



```
Shiva@DESKTOP-MRCVEKT: ~/class7
Shiva@DESKTOP-MRCVEKT:~/class7$ mkdir mydir
Shiva@DESKTOP-MRCVEKT:~/class7$ ln mydir dirlink
ln: mydir: hard link not allowed for directory
Shiva@DESKTOP-MRCVEKT:~/class7$ ln -s mydir dirlink
Shiva@DESKTOP-MRCVEKT:~/class7$ ls -ld dirlink
lrwxrwxrwx 1 Shiva Shiva  5 Oct  1 18:39 dirlink -> mydir
Shiva@DESKTOP-MRCVEKT:~/class7$
```

Screenshot 8: a hard link on a directory is not allowed, a soft link is.

My teacher said hard links are not allowed for directories because it would be easy to make a loop, where a folder ends up inside itself, and then commands that walk through the folders would go round and round for ever.

What happens if the original is deleted

This is the test that shows the real difference between the two. I deleted data.txt and then tried to read both links:

```
rm data.txt
cat hardlink.txt
cat softlink.txt
```

The hard link still printed all five lines, because it was never a copy, it was the file itself with another name. The file is only really removed when the last name is gone.

The soft link failed with `cat: softlink.txt: No such file or directory`. It was only holding the name `data.txt`, and that name does not exist any more, so the link points at nothing. In the listing it even turned red to show that it is broken.

```
Shiva@DESKTOP-MRCVEKT: ~/class7
Shiva@DESKTOP-MRCVEKT:~/class7$ rm data.txt
Shiva@DESKTOP-MRCVEKT:~/class7$ cat hardlink.txt
Line one of my practice file.
Line two is here.
Line three.
Line four.
Line five is the last one.
Shiva@DESKTOP-MRCVEKT:~/class7$ cat softlink.txt
cat: softlink.txt: No such file or directory
Shiva@DESKTOP-MRCVEKT:~/class7$ ls -li
total 12
4579 lrwxrwxrwx 1 Shiva Shiva 5 Oct 1 18:39 dirlink -> mydir
2930 -rw-r--r-- 1 Shiva Shiva 98 Oct 1 18:38 hardlink.txt
4578 drwxr-xr-x 2 Shiva Shiva 4096 Oct 1 18:39 mydir
4479 -rw-r--r-- 1 Shiva Shiva 98 Oct 1 18:34 notes.txt
4577 lrwxrwxrwx 1 Shiva Shiva 8 Oct 1 18:38 softlink.txt -> data.txt
Shiva@DESKTOP-MRCVEKT:~/class7$
```

Screenshot 9: the hard link still works after the original is deleted, the soft link does not.

The count also went back from 2 to 1, because only one name is left.

This is the difference side by side:

	Hard link (ln)	Soft link (ln -s)
Inode number	Same as the original.	Its own, different one.
Size	Same as the original.	Only a few bytes.
ls -l shows	A normal file, starts with -	Starts with l and shows an arrow.
Works on a directory	No.	Yes.
If the original is deleted	Still works.	Breaks.

One last thing I noticed. A hard link cannot be made across two different drives, because the inode number only makes sense inside one filesystem. A soft link can, because it is only holding a name.

Commands I used

Command	What it does
<code>cat file.txt</code>	Prints the whole file at once.
<code>more file.txt</code>	Shows one screen at a time, only forwards.
<code>less file.txt</code>	Shows one screen at a time, forwards and backwards.
<code>groups</code>	The groups I myself belong to.
<code>groups arjun</code>	The groups the user arjun belongs to.

Command	What it does
<code>id arjun</code>	The same thing with the id numbers as well.
<code>getent group devops</code>	The other way round: who is inside this group.
<code>free</code>	Memory usage in kilobytes.
<code>free -h</code>	The same numbers in GB and MB.
<code>chgrp users file1.txt</code>	Changes the group of the file, owner stays.
<code>ln data.txt hardlink.txt</code>	Makes a hard link.
<code>ln -s data.txt softlink.txt</code>	Makes a soft link.
<code>ls -li</code>	Listing with the inode number in front.